past the end of the array, you just get the end of the array, you don't get blank elements or anything like that. If the length is negative, you get nil, and if the first element is exactly the length of the string, you get an empty array.

- A range, in which case you get the subarray starting at the index represented by the first element of the range and ending at the index represented by the last element of the range. Again, negative numbers are counted from the end of the array, meaning you can get weird things like [0, 1, 2, 3][-3..2], which returns [1, 2] because -3 is the third element from the end, and 2 is farther in the array than that. If the start of the range is outside the array, you get nil. If the end of the range is outside the array, you get elements up until the end of the array. Endless ranges work too, so [...-1] will give you everything but the last element of the array.

All the square bracket behavior is aliased as the method slice. The single argument behavior also works as the method at. The behavior of both of hese methods is to return nil if you completely go outside the bounds of the array.

If you want different behavior, the fetch method behaves like at, taking a single index argument, but fetch takes an optional second argument that can either be a positional argument or a block. If the index is out of range, the behavior of fetch depends on which argument is used, either the positional argument is returned or the block is called with the index argument and the block's value is returned.

Common use cases have shortcuts: first and last return the first or last element of the array, respectively. An optional argument allows you to specify more than one return value; if the optional argument is 0, you get an empty array in response. The method values_at takes an arbitrary number of index arguments and returns a new array of the values at those indexes. The method dig takes an arbitrary number of arguments, returning the value at that index and passing that value to the message dig with the remaining arguments. For example:

```
array = [1, 2, 3, 4, [5, 6], 7]
array[2]                                  # => 3
array[-3]                                 # => 4
array[1, 3]                               # => [2, 3, 4]
array[2..4]                               # => [3, 4, [5, 6]]
array.fetch(10, 100)                      # => 100
array.fetch(10) { |i| "No value at #{i}" } # => "No value at 10"
array.first                               # => 1
array.first(3)                            # => [1, 2, 3]
array.last                                # => 7
array.last(2)                             # => [[5, 6], 7]
array.values_at(2, 4, 6)                  # => [3, [5, 6], nil]
array.dig(-2, 1)                          # => 6
```

Ruby also allows you to search an array by value. The method index takes an argument or a block. With an argument, it returns the index of the first element that is equal to the argument, or nil if there is no element. With a block, index returns the index of the first element that causes the block to return a truthy value. If you want the last element, the method rindex does the same thing, but starts from the end of the array.